# Introduction to Computer Vision and Convolutional Neural Network

If you've been following this series then you already know about regression and classification using `TensorFlow`. In this article, we will learn about `Computer Vision` and `Convolutional Neural Network` (CNN).

**You can find me on**

- LinkedIn,
- YouTube,
- Gmail,
- discord,

**You can find all my articles and code on**

- Deep Learning With Tensorflow,

**If you are new to Machine learning, I also have a github for beginner machine learning.**

- Machine Learning Zero to Hero

**Pre-requisites**

- Python.
- Numpy.
- Pandas.
- Matplotlib/Seaborn.
- Basics of data analysis and preprocessing.
- Basics ML terminologies.
- Basics Tensorflow and keras terminologies.
- Basics of Neural Network.
- Read the previous article on Multi Class Classification.

**Installations**

We need to install Tensorflow. To do so, we will use the following command:

```
pip install tensorflow
```

or If you have conda installed,

```
conda install tensorflow
```

# What is Computer Vision?

It's exactly as it sounds. We make computer see and recognise stuff.

We do that using the power of `Machine learning`.

We can use computer vision for many useful tasks like: Autonomous driving, face recognition, Healthcare, Manufacturing and many more.

But in machine learning lingo these task can be summarised as:

- Image Classification.
- Object detection.
- Segmentation.

And in the core of all of them, we use `Convolutional Neural Network` (CNN).

# Convolutional Neural Network

CNN is not like the typical layered perceptrons model or a normal feedforward network. It uses `convolution` to learn the pattern in the data. And it is a `deep learning model` which is trained using `backpropagation algorithm`.

So, we are changing the recipe.

Let's start with image classification.

## Image Classification

If you've read my previous articles, you already know that we can make a feedforward Neural Network to classify images by flattening the image and then passing it to the Neural Network. But there are some problems.

...But there are some problems.

### The Problem with Flattening

When we pass an image into a traditional feedforward network (or Multi-Layer Perceptron), we have to **flatten** the 2D or 3D grid of pixels into a single, long 1D vector.

For example, a small color image of $64 \times 64$ pixels has $64 \times 64 \times 3 \text{ (RGB channels)} = 12,288$ individual values. If we feed this flat vector directly into a standard Dense layer, we run into two massive roadblocks:

1. **Lack of Spatial Awareness (No Inductive Bias):** Images are defined by structure—pixels that sit next to each other form edges, edges form shapes, and shapes form objects. While flattening doesn't actually erase or "destroy" these pixel values, a conventional Dense layer treats every input node independently. It lacks an inherent understanding of 2D grid topology, meaning it doesn't automatically exploit local neighbor relationships and has to learn them from scratch for every single position.

2. **The Parameter Explosion:** For tiny images like $16 \times 16$, this is fairly manageable. But for our $64 \times 64$ example, if our first hidden layer has just 50 neurons, that single weight matrix requires over **600 thousand weights** $(12,288 \times 50)$. Scale that up to a standard smartphone photo, and the parameter count spirals out of control, leading to massive memory usage and severe overfitting.

This is why we change the recipe. Enter the **Convolutional Neural Network (CNN)**.

# How CNNs Fix This: The Core Ingredients

Instead of treating an image like a random list of numbers, a CNN treats an image like an actual map. It solves the problems above by using a sequence of specialized layers to automatically extract features without destroying the original shape of the data.

Here is how a standard CNN architecture builds its "recipe" from start to finish:

```
[Input Image] → [Convolution Layer + ReLU] → [Pooling] → [Flatten] → [Fully Connected Layer] → [Out
```

## 1. The Convolution Layer (The Feature Finder)

The core building block of a CNN is the **Convolutional Layer**. Instead of connecting every pixel to every single neuron, it uses small matrices called **filters** or **kernels** (typically $3 \times 3$ or $5 \times 5$ grids).

This filter slides (or *convolves*) across the image piece by piece. At each stop, it performs basic matrix multiplication to scan for specific patterns like vertical edges, horizontal lines, or curves. The output of this sliding process is a **Feature Map**, which highlights exactly where those patterns exist in the image.



## The Learnable Parameters

It is crucial to understand that **the numbers inside these filters are the actual weights (parameters) of the network**. In traditional feedforward networks, weights are assigned to individual connections between neurons. Here, the weights live inside the filter matrix. Just like traditional weights, these filter values are initialized randomly and are **gradually optimized by the backpropagation algorithm** using gradient descent as the network learns from its mistakes.

## How the Operation Works

This filter slides (or *convolves*) across the width and height of the input image piece by piece. At each stop, it performs an element-wise multiplication between the filter's weights and the current chunk of pixel values it is covering, summing up the results into a single value.

The output of this entire sliding process is a new grid called a **Feature Map**, which highlights exactly where specific patterns exist in the image.

- **Automatic Feature Engineering:** In early computer vision, engineers manually hardcoded these filter values to detect specific things (like a Sobel filter for vertical edges). In a CNN, we don't hardcode anything. The backpropagation algorithm automatically discovers *what* features are important. Early layers might learn filters that detect basic edges, while deeper layers automatically learn filters that look for complex shapes like eyes or tires.
- **Parameter Sharing:** The magic here is that the exact same filter is used to scan the entire image. If a filter learns to detect a horizontal edge, it can find that edge whether it is at the top, bottom, or center of the image.

This drastically cuts down the number of parameters the network needs to optimize.

## 2. The Activation Function (Adding Non-Linearity)

Real-world visual data is highly complex and non-linear. The convolution operation described above is entirely linear—it is just repeated multiplication and addition. If we stacked multiple convolutional layers without an activation function, the entire network would mathematically collapse into just one single, giant linear equation, making it incapable of learning intricate patterns.

To prevent this, right after the convolution operation, the network passes the feature maps through a non-linear activation function, most commonly **ReLU (Rectified Linear Unit)**.

Mathematically, ReLU is incredibly simple:

$$f(x) = \max(0, x)$$

It leaves any positive value completely unchanged but turns any negative value into a hard zero.

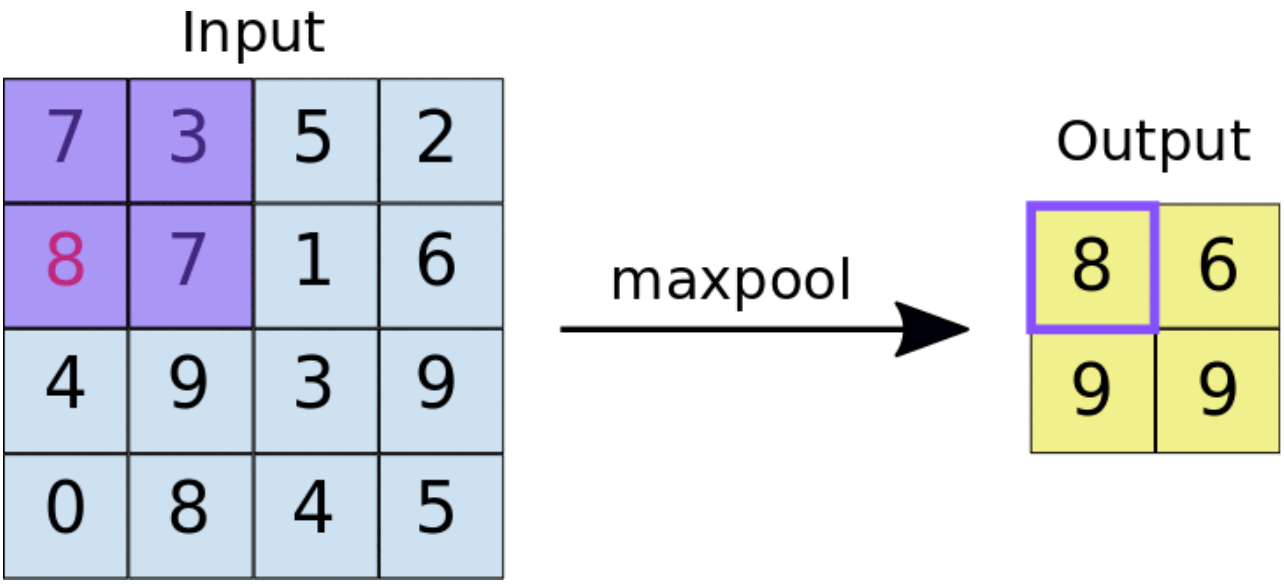
By clamping negative activations to zero, ReLU introduces non-linearity into the network. This allows the CNN to learn complex, non-linear boundaries and ensures that neurons only fire when they detect a relevant feature, creating a sparse and computationally efficient representation of the image.
Use code with caution.

## 3. The Pooling Layer (The Downsampler)

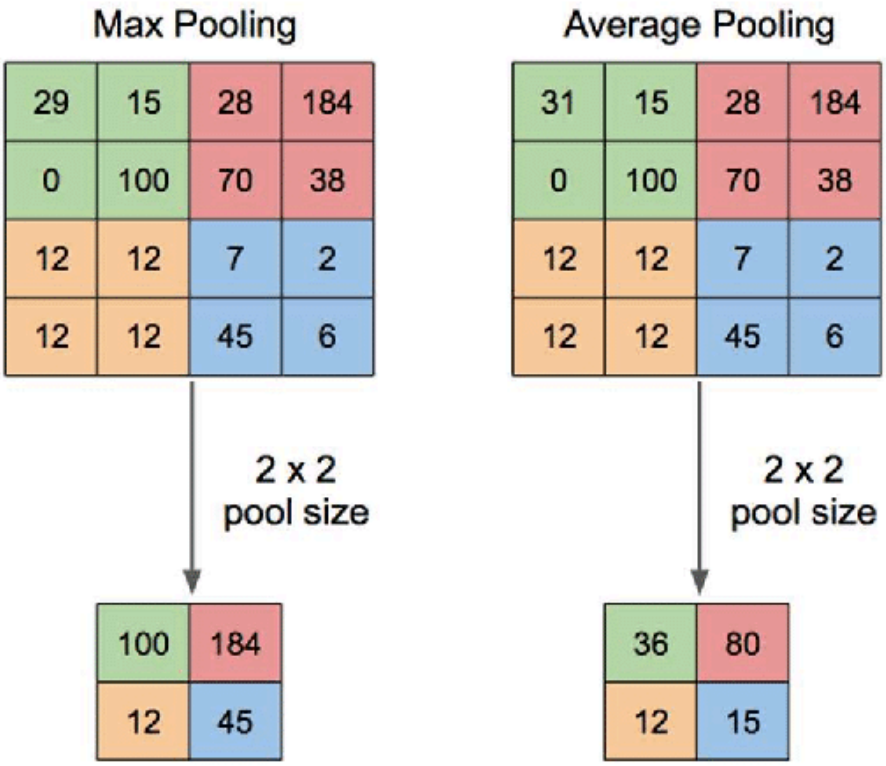
Images contain a lot of redundant information. A **Pooling Layer** is introduced to reduce the spatial size (width and height) of the feature maps.

The most common method is **Max Pooling**, where a small window (e.g., $2 \times 2$) slides across the feature map and keeps only the maximum value from that region. This does two crucial things:

- **It shrinks the data size:** By cutting the width and height down, it keeps the network computationally light, fast, and memory-efficient.
- **It provides local spatial tolerance:** If a feature shifts slightly by a pixel or two, the local max value often remains the same. This gives the network robustness against minor local jitter and small spatial shifts. *(Keep in mind: while local pooling helps with small shifts, handling major position changes—like recognizing an object anywhere from the top-left corner to the bottom-right—is built up gradually by stacking multiple convolutional and pooling layers together, rather than a single pooling step).*




I like `Average pooling` , the concept is almost exactly the same except instead of taking the maxing of a region it takes average.




## 4. Fully Connected Layer (The Classifier)

Then we will need a fully connected layer.

By stacking multiple Convolution and Pooling layers together, the network learns a hierarchy of features: early layers detect simple edges, middle layers detect textures and shapes, and deep layers recognize whole objects (like eyes, noses, or wheels).

Finally, once the features are extracted, the final feature map is **flattened** into a vector. It is then passed to a traditional **Fully Connected (Dense) Layer**. This layer acts as the final judge, looking at all the gathered features to calculate the probability of what the image actually is (e.g., Cat: 92%, Dog: 5%, Car: 3%).

Alright! that covers the main theory behind Convolutional Neural Networks. Now let's get to the actual code.

Let's make a CNN model

# Getting the data

As we are testing out the theory I'll just use a very simple dataset.

after that follow the steps below.

```
import tensorflow as tf

print(tf.config.list_physical_devices("GPU"))
```

```
2026-10-21 15:53:24.405409: I tensorflow/core/platform/cpu_feature_guard.cc:210] Thi
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorF

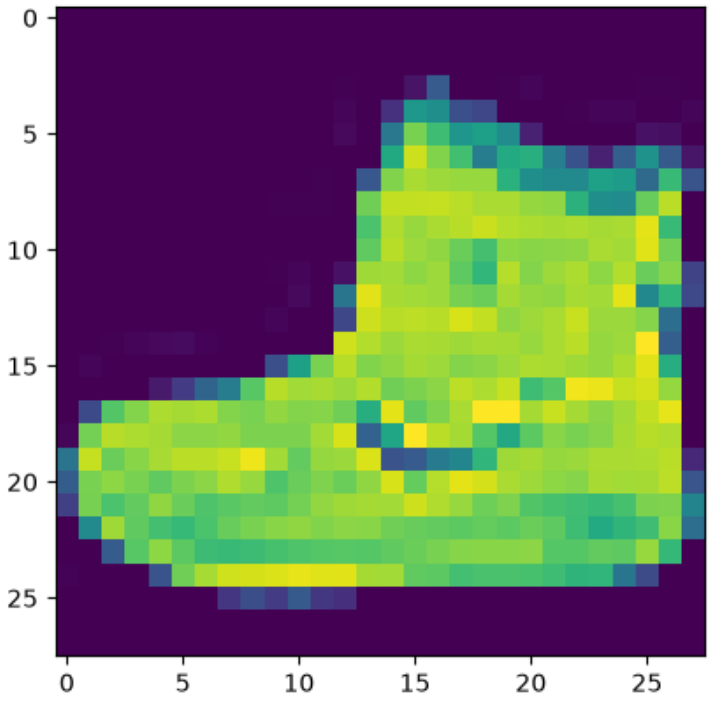

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
from keras.datasets import fashion_mnist
```

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

```
from matplotlib import pyplot as plt

plt.imshow(x_train[0])
plt.show()
```




The above dataset is called the **Fashion MNIST** dataset. It is a alternative version of the classic MNIST dataset. It contains 10 different classes of clothing items.

Let's see what the classes are.

```
1 import numpy as np
2
3 np.unique(y_train)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], dtype=uint8)
```

So, the classes are already set as `0-9` numeric values but whats the fun in that?

You can find the labels here: https://github.com/zalandoresearch/fashion-mnist#labels



We can save these labels as an array or a dictionary. Either way is fine.

```
labels = [
    "T-shirt",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot",
]
```

This will work because if we make a classification model in the output we will get a `probability array` for each class `0-9` sequentially. So, each index of the probability array will represent the class and the original label also be in the same index.

# Creating the model using TensorFlow

Remember the architecture of a Convolutional Neural Network?

```
Input -> Convolution -> Pooling -> flatten -> Fully Connected -> Output
```

Now let's create the model using TensorFlow.

```
from keras import layers, models
```

```
model = models.Sequential(
    [
        layers.Input((28, 28, 1)),
        layers.Conv2D(5, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(5, activation="relu"),
        layers.Dense(5, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ]
)

model.summary()
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDE
I0000 00:00:1792576406.791379  340505 gpu_device.cc:2020] Created device /job:localh
```

**Model: "sequential"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 5) | 50 |
| average_pooling2d (AveragePooling2D) | (None, 13, 13, 5) | 0 |
| flatten (Flatten) | (None, 845) | 0 |
| dense (Dense) | (None, 5) | 4,230 |
| dense_1 (Dense) | (None, 5) | 30 |
| dense_2 (Dense) | (None, 10) | 60 |

**Total params:** 4,370 (17.07 KB)

**Trainable params:** 4,370 (17.07 KB)

```
Non-trainable params: 0 (0.00 B)
```

> I used `AveragePooling2D` instead of `MaxPooling2D`, because I don't like when data is lost.

Now, we can compile and train the model.

```
model.compile(
    optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"]
)
```

Before training we must normalize the data.

```
x_train = x_train / 255
x_test = x_test / 255
```

```
model.fit(x_train, y_train, epochs=5)
```

```
Epoch 1/5


2026-10-21 15:53:28.559609: I external/local_xla/xla/service/service.cc:163] XLA ser
2026-10-21 15:53:28.559621: I external/local_xla/xla/service/service.cc:171]   Strea
2026-10-21 15:53:28.576608: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_ut
2026-10-21 15:53:28.695346: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.c
2026-10-21 15:53:29.445980: I external/local_xla/xla/stream_executor/cuda/subprocess

2026-10-21 15:53:29.491672: I external/local_xla/xla/stream_executor/cuda/subprocess



 107/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.1384 - loss: 2.1941

I0000 00:00:1792576410.644572  340565 device_compiler.h:196] Compiled cluster using


1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 914us/step - accuracy: 0.6370 - loss: 1.0072
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 853us/step - accuracy: 0.7661 - loss: 0.6432
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 855us/step - accuracy: 0.7855 - loss: 0.5861
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 874us/step - accuracy: 0.7981 - loss: 0.5621
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 866us/step - accuracy: 0.8066 - loss: 0.5433




<keras.src.callbacks.history.History at 0x72f9003a7cb0>
```

We have a fairly modest model. At least it's not less than 50%.

Now let's break down the model we made.

# Putting It Into Code: A Real-World Example

Let's look at how we build this recipe in code using Keras/TensorFlow. Here is a sequential CNN model designed to take in grayscale images (like the famous MNIST handwritten digit dataset) and classify them into 10 distinct categories:

```
model = models.Sequential(
    [
        layers.Input((28, 28, 1)),
        layers.Conv2D(5, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(5, activation="relu"),
        layers.Dense(5, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ]
)
```

Let's break this down line-by-line to see exactly what is happening to our data at every step:

## 1. `layers.Input((28, 28, 1))`

This defines the entry point for our data. The model expects an input image with a shape of **28 × 28 × 1**.

- The `28, 28` represents the width and height of the image in pixels.
- The `1` represents the number of color channels. A `1` means it is a grayscale image (if it were a color RGB image, this number would be `3`).

## 2. `layers.Conv2D(5, (3, 3), activation='relu')`

This is where our feature extraction begins! We apply a **2D Convolutional Layer** with the following settings:

- `5`: This tells the network to create **5 different filters**. Each filter will learn to look for a unique feature (like horizontal lines, diagonals, or curves).
- `(3, 3)`: The spatial size of each filter matrix. It will scan the image in tiny 3 × 3 pixel patches.
- `activation='relu'`: As we discussed earlier, immediately after convolving, every negative value in our feature maps is turned to zero to introduce non-linearity.
- *Output shape after this layer:* Since a 3 × 3 filter cannot perfectly center on the very outer edges of a 28 × 28 image without padding, the output size shrinks slightly to **26 × 26 × 5** (5 feature maps, each 26 × 26 pixels).

## 3. `layers.AveragePooling2D((2, 2))`

Next, we downsample the data using an **Average Pooling Layer** instead of the traditional Max Pooling.

- The `(2, 2)` grid slides across our 5 feature maps. Instead of picking only the single highest value, it calculates the **mathematical average** of all four pixel values within that 2 × 2 area.
- While Max Pooling focuses purely on the sharpest, most dominant features (like bright edges), Average Pooling smooths things out by retaining a blend of all the background and feature information in that region.
- This operation cuts the width and height of our feature maps exactly in half.
- *Output shape after this layer:* The dimensions drop from 26 × 26 × 5 down to **13 × 13 × 5**. We have successfully compressed the image data while preserving a smooth, summarized version of its features!

### 4. `layers.Flatten()`

Our spatial feature extraction is complete. Now it is time to pivot to the classification phase. The `Flatten()` layer takes our 3D tensor of feature maps (13 × 13 × 5) and unrolls it into a single, long **1D array**.

- *Math check:* 13 × 13 × 5 = 845.
- *Output shape after this layer:* A single flat vector containing **845 individual values**, ready to be fed into a standard neural network.

### 5. `layers.Dense(5, activation='relu')`

This is our first **Fully Connected (Dense) Layer**. Every single one of the 845 flattened values is connected to the **5 neurons** in this layer. This layer begins the high-level reasoning, combining the extracted features to look for broader patterns. We use the `'relu'` activation function here again to keep the relationships non-linear.

### 6. `layers.Dense(5, activation='relu')`

Another **Dense Layer** with **5 neurons** and a `'relu'` activation. Stacking dense layers allows the network to learn deeper, more abstract combinations of features before making its final guess.

### 7. `layers.Dense(10, activation='softmax')`

This is the **Output Layer**. Because our goal is to classify the image into 1 of 10 possible classes, this layer has exactly **10 neurons**.

- Instead of ReLU, we use the `'softmax'` activation function here. Softmax takes the raw outputs from the 10 neurons and squashes them into a **probability distribution** between 0 and 1, ensuring all 10 outputs add up to exactly 100%.
- The neuron with the highest probability value is the model's final prediction (e.g., "There is a 94% chance this image is the number 7").

And now we have a model with fewer parameters but very powerful in pattern recognition.

Well, that's the image classification with `convolutional neural network` for you.

That was just to show you how a convolutional neural network works and how usefull it can be.

Now time to a proper example.

# Loading Real images

Luckily if you need huge ammount image data, there is a fully free image dataset called Food-101.

> The official website contain the whole dataset but I'll not be using all of it. I'm going to modify and keep three food category and images.

I've taken two food classes from the whole dataset made a folder named `cheese_cake_pizza` you can download the dataset urself from my here.

**You can uncomment the following cell to download and extract the dataset.**

```
# !wget "https://github.com/RishatTalukder/Deep_Learning_With_Tensorflow/raw/refs/heads/main/

# import zipfile

# zip_ref = zipfile.ZipFile('cheese_cake_pizza.zip')
# zip_ref.extractall(path='resource')
# zip_ref.close()
```

After running the above code, you should have a folder named `cheese_cake_pizza` inside a folder named `resource` in your current directory and the zip file too.

Now let's take a look at the extracted folder.

```
!ls resource/cheese_cake_pizza
```

```
test  train  val
```

Hmmm, there's three folders. `test`, `train` and `val`. Let's take a look at the `train` folder.

```
!ls resource/cheese_cake_pizza/train
```

```
cheesecake  pizza
```

Two folders: cheese_cake and pizza

What about the other two folders?

```
!ls resource/cheese_cake_pizza/test
```

```
cheesecake  pizza
```

```
!ls resource/cheese_cake_pizza/val
```

```
cheesecake  pizza
```

So, inside each of these folders there are two folders. `cheese_cake` and `pizza`. Whats inside the `cheese_cake` folder?

```
!ls resource/cheese_cake_pizza/train/cheesecake
```

```
1001446.jpg  1690850.jpg  2240092.jpg  2788779.jpg  3369593.jpg  3920168.jpg
1004515.jpg  1692172.jpg  2240302.jpg  2788868.jpg  3377918.jpg  3923783.jpg
1004807.jpg  1693775.jpg  2241264.jpg  2794584.jpg  3388138.jpg  399407.jpg
1021942.jpg  1694199.jpg  2243942.jpg  280106.jpg   3399394.jpg  402953.jpg
1023543.jpg  1703386.jpg  2245899.jpg  2801570.jpg  3405731.jpg  403610.jpg
1034351.jpg  1707618.jpg  2249472.jpg  2803707.jpg  3406925.jp

.....

165910.jpg   2229007.jpg  2773483.jpg  3344906.jpg  3900431.jpg  998656.jpg
1667024.jpg  2230316.jpg  2780839.jpg  3345639.jpg  3904984.jpg
1678182.jpg  2230878.jpg  2781710.jpg  3351068.jpg  3915954.jpg
1679654.jpg  2233012.jpg  2785771.jpg  335935.jpg   391930.jpg
1681931.jpg  2236408.jpg  2788336.jpg  3361907.jpg  3919885.jpg
```

Ow wow! a lot of images! It would be nice to see the folder structure with how many images are in each folder.

```
import os

for dirpath, dirnames, filenames in os.walk("resource/cheese_cake_pizza"):
    print(
        f"There are {len(dirnames)} directories and {len(filenames)} images in '{dirpath}'."
    )
```

```
There are 3 directories and 0 images in 'resource/cheese_cake_pizza'.
There are 2 directories and 0 images in 'resource/cheese_cake_pizza/train'.
There are 0 directories and 800 images in 'resource/cheese_cake_pizza/train/cheeseca
There are 0 directories and 800 images in 'resource/cheese_cake_pizza/train/pizza'.
There are 2 directories and 0 images in 'resource/cheese_cake_pizza/val'.
There are 0 directories and 100 images in 'resource/cheese_cake_pizza/val/cheesecake
There are 0 directories and 100 images in 'resource/cheese_cake_pizza/val/pizza'.
There are 2 directories and 0 images in 'resource/cheese_cake_pizza/test'.
There are 0 directories and 100 images in 'resource/cheese_cake_pizza/test/cheesecak
There are 0 directories and 100 images in 'resource/cheese_cake_pizza/test/pizza'.
```

Well, from the above cell we can see that In the train directory there are 2 directories with 800 images each, a total of 1600 images.

Test directories have 100 images each, a total of 200 images.

For validation directory it's the same as test directory, a total of 200 images.

I organised these folders to represent the train, test and validation sets.

So, let's see how they look like.

# Loading the training set

```
from keras.utils import image_dataset_from_directory

image_dataset = image_dataset_from_directory(
    "resource/cheese_cake_pizza/train", labels="inferred", image_size=(224, 224), seed=1
)

image_dataset
```

```
Found 1600 files belonging to 2 classes.

<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float
```

`image_dataset_from_directory` returns a `tf.data.Dataset` object. This single function will do a lot of the heavy lifting for us. It will automatically can read the images from a directory, for my case `resource/cheese_cake_pizza/train` folder and will automatically label the images based on the folder names.

> One good thing is, it does not load the images into memory. It will read the images from the disk and process them on the fly and only load the images into memory when we need them.

There are a lot of arguments we can pass to the `image_dataset_from_directory` function.

By default you have to pass `directory` argument. It is the path to the directory that contains the images.

I passed:

- `Directory`: `resource/cheese_cake_pizza/train`
- `Labels`: `'inferred'` meaning the dataset will automatically label the images based on the folder names.
- `Image Size`: `(224,224)` meaning the images will be resized to this size. We have to do this because the convolutional neural network is trained on images of a specific size.
- `seed`: `1` meaning the dataset will be shuffled randomly but in the same order every time we run the code. This helps ensure reproducibility.

So, let's explore our dataset.

Are the class names correct?

```
image_dataset.class_names
```

```
['cheesecake', 'pizza']
```

Looks like it.

Let's plot some images and see there labels.

```
import tensorflow as tf

images, labels = image_dataset.as_numpy_iterator().next()
```

Let's break down the code above first.

As `image_dataset` is a `tf.data.Dataset` object, it lazy loads the images from the disk. So, to completely load the images into memory, we need to use the `as_numpy_iterator` method.

This will return an `iterator` object that will allow us to access the image `batches` one by one.

> NOTE: The images are loaded as batches of 32 images at a time. Because if you load all the images directly into memory, you'll can run out of ram and the whole thing will crash.

We can take out each batch with the `next()` method which will return a tuple of images and their corresponding labels.

Now, to see if everything is loaded properly, let's print the shape of the images variable.

```
images.shape
```

```
(32, 224, 224, 3)
```

As you can see the shape is `(32, 224, 224, 3)` which means there are 32 images with a shape of `(224, 224, 3)`.

We can count how many batches with a simple for loop.

```
batch_count = 0

for images, labels in image_dataset.as_numpy_iterator():
    batch_count += 1
    print(f"Number of images in batch {batch_count}: {len(images)}")

print(f"Number of batches: {batch_count}")
```

```
Number of images in batch 1: 32
Number of images in batch 2: 32
Number of images in batch 3: 32
Number of images in batch 4: 32
....
Number of images in batch 49: 32
Number of images in batch 50: 32
Number of batches: 50


2026-10-21 15:53:41.519638: I tensorflow/core/framework/local_rendezvous.cc:407] Loc
```

As you can see the whole dataset has 50 batches of 32 images each.

With some simple math we can calculate the total number of images in the dataset.

```
1 | 50 * 32
```

```
1600
```

Exactly 1600!

That right!

Now, let's see the shape of the first image.

```
1 | images[0].shape
```

```
(224, 224, 3)
```

The shape of the first image is `(224, 224, 3)` which means it is a 224x224 image with `3 color channels`.

Unlike the mnist dataset, the images are not one grayscale image but 3 color channels. This is the RGB values of each pixel and it is set as an array of 3 values for each pixel.

Let's see the range of the color values.

```
1 | np.max(images[0]), np.min(images[0])
```

```
(np.float32(255.0), np.float32(0.0))
```

Minimum is `0.0` and maximum is `255.0`.

That's just like before right?

Yes, But there is an issue. Take a closer look at the data type of the color values and try figure out what is different from the mnist dataset images.

It's `float32`, in the mnist dataset it was `uint8` which is an unsigned integer with a range of 0 to 255.

We are going to plot the images with the `matplotlib` library. And in matplotlib `imshow` function accepts the color value to be floating point only if it is between `0.0` and `1.0`.

So, how do We make any value between `0.0` and `1.0`? We standardise the values by dividing them by `255` just like before.

```
images = images / 255
```

```
np.max(images[0]), np.min(images[0])
```

```
(np.float32(1.0), np.float32(0.0))
```

And now we can plot the images.

Before that one last thing. We should inspect the labels.

```
labels
```

```
array([0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1,
       1, 0, 1, 0, 1, 0, 0, 0, 1, 0], dtype=int32)
```

As you can see all the labels are either 0 or 1. But which one is cheese cake and which one is pizza?

> Always remember the labels are the index of the classnames.

So, what are the class names?

```
image_dataset.class_names
```

```
['cheesecake', 'pizza']
```

Index `0` is `cheesecake` and index `1` is `pizza`.

So,

Label `0` = `cheesecake`
Label `1` = `pizza`

Now let's visualize

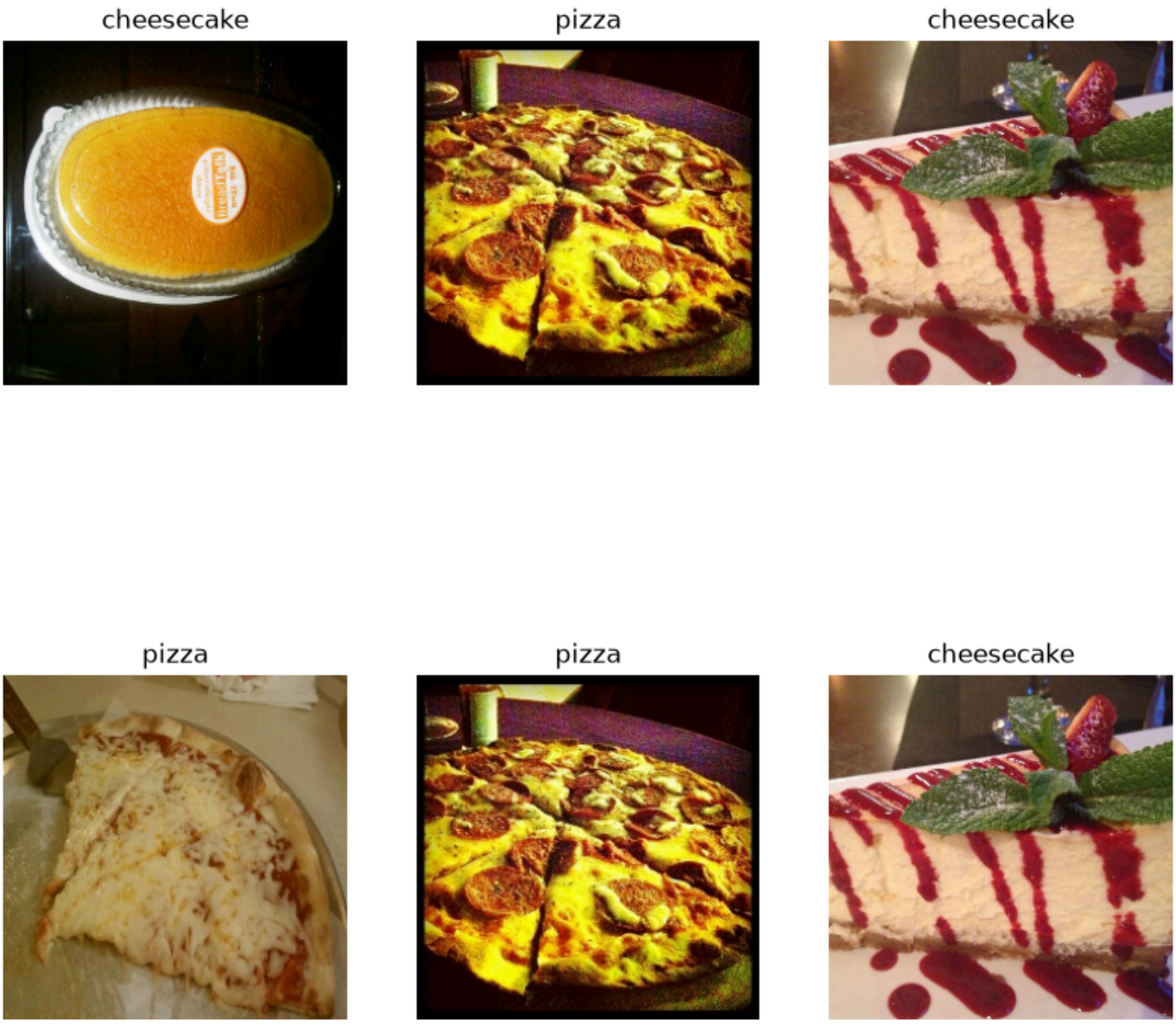
```python
import random


def plot_randon_images(images, labels):
    plt.figure(figsize=(10, 10))

    for i in range(6):
        ax = plt.subplot(2, 3, i + 1)
        rand_index = random.choice(range(len(images)))
        plt.imshow(images[rand_index])
        label_ind = labels[rand_index]
        plt.title(image_dataset.class_names[label_ind])
        plt.axis("off")

    plt.show()
```

```python
plot_randon_images(images, labels)
```



Ow, look at that!

Look really nice, right? Some the images are repeated because the batch only has 32 images.

So, the data is loading exactly how we want it to be and also we have to remember that we must standardise the images.

So, let's properly load all the datasets.

```
training_dataset = image_dataset_from_directory(
    directory="resource/cheese_cake_pizza/train",
    image_size=(224, 224),
)

testing_dataset = image_dataset_from_directory(
    directory="resource/cheese_cake_pizza/test", image_size=(224, 224), shuffle=False
)

validation_dataset = image_dataset_from_directory(
    directory="resource/cheese_cake_pizza/val", image_size=(224, 224), shuffle=False
)
```

```
Found 1600 files belonging to 2 classes.
Found 200 files belonging to 2 classes.
Found 200 files belonging to 2 classes.
```

Well, with that the data is ready to go.

> Here, the validation set and the testing sets suffle argument is set to `False` . Just remember that, This has a reason and we will come back to it later.

We should make a model.

# Model

WE can use the precious model in this article but where's the fun in that. We have real images and each image will have complex patterns. So, we have to make a more complex model.

Also remember the is not leaded into memory yet, we have to scale the images to be between `0.0` and `1.0` .

```python
model = models.Sequential(
    [
        layers.Input(shape=(224, 224, 3)),
        layers.Rescaling(1.0 / 255),  # 0-255 to 0-1
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((4, 4)),
        layers.Flatten(),
        layers.Dense(5, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ]
)

model.summary()
```

**Model: "sequential_1"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| rescaling (Rescaling) | (None, 224, 224, 3) | 0 |
| conv2d_1 (Conv2D) | (None, 222, 222, 10) | 280 |
| conv2d_2 (Conv2D) | (None, 220, 220, 10) | 910 |
| average_pooling2d_1 (AveragePooling2D) | (None, 110, 110, 10) | 0 |
| conv2d_3 (Conv2D) | (None, 108, 108, 10) | 910 |
| conv2d_4 (Conv2D) | (None, 106, 106, 10) | 910 |
| average_pooling2d_2 (AveragePooling2D) | (None, 26, 26, 10) | 0 |
| flatten_1 (Flatten) | (None, 6760) | 0 |
| dense_3 (Dense) | (None, 5) | 33,805 |
| dense_4 (Dense) | (None, 1) | 6 |

**Total params:** 36,821 (143.83 KB)

**Trainable params:** 36,821 (143.83 KB)

```
Non-trainable params: 0 (0.00 B)
```

Let's break it down.

**1. Input Layer and Normalization**

```
(layers.Input(shape=(224, 224, 3)),)
(layers.Rescaling(1.0 / 255),)
```

- Input layer: Each image has a height of 224 pixels, a width of 224 pixels, and 3 color channels (Red, Green, Blue).
- Rescaling layer: Divides every pixel value by 255, converting pixel intensities from the range $([0, 255])$ to $([0, 1])$.

**2. First Convolutional Block**

```
(layers.Conv2D(10, (3, 3), activation="relu"),)
(layers.Conv2D(10, (3, 3), activation="relu"),)
(layers.AveragePooling2D((2, 2)),)
```

This block extracts low-level visual features from the input image.

- First convolution: Applies 10 learnable filters of size $(3 \times 3)$ to detect patterns such as edges, corners, and simple textures.
- Second convolution: Applies another 10 filters to learn more complex combinations of the features detected by the first convolution.
- Average pooling: Uses a $(2 \times 2)$ window to reduce the spatial dimensions by calculating the average of each local region.

Because no padding is specified, the convolutional layers use `padding='valid'` by default. This means the feature maps become smaller after each convolution.

| Stage | Output shape |
|---|---|
| Input | $(224 \times 224 \times 3)$ |
| Rescaling | $(224 \times 224 \times 3)$ |
| Conv2D (10 filters) | $(222 \times 222 \times 10)$ |
| Conv2D (10 filters) | $(220 \times 220 \times 10)$ |
| AveragePooling2D | $(110 \times 110 \times 10)$ |

The ReLU activation function introduces non-linearity, allowing the network to learn more complex patterns.

$$\text{ReLU}(x) = \max(0, x)$$

Now you might ask why does the feature map after the first layer have dimensions $222 \times 222 \times 10$.

## First, why is the output $222 \times 222$ instead of $224 \times 224$?

To understand this, we first need to understand **padding**.

Remember that our convolutional filter has a spatial size of $3 \times 3$. When we apply this filter to an image, it needs a complete $3 \times 3$ region to calculate each output value.

Now, here's a question: **Can we place a $3 \times 3$ filter at the very edge of an image and still keep the entire filter inside the image?**

Not always! If we place the filter at the top-left corner, for example, it fits perfectly. But as we move it toward the right and bottom edges, we eventually run out of pixels to fill the entire filter.

As a result, without padding, the output becomes smaller than the original image.

So, what can we do to prevent this?

We can add extra pixels around the borders of the image. This technique is called **padding**. Usually, these extra pixels are filled with zeros, which is why this approach is called zero padding.

If we use `padding='same'`, TensorFlow adds padding as needed to preserve the spatial dimensions when the stride is 1.

However, we haven't specified any padding in our model:

```
layers.Conv2D(10, (3, 3), activation="relu")
```

By default, Keras uses `padding='valid'`, which means **no padding is added**.

For a convolution with stride 1 and a $3 \times 3$ filter, the output dimension is calculated as:

$$\text{Output size} = \text{Input size} - \text{Filter size} + 1$$

Therefore, our output height and width become:

$$224 - 3 + 1 = 222$$

So, the output's spatial dimensions are $222 \times 222$.

## But wait, what about the three RGB channels?

You might now ask: **If our input image has three color channels, why doesn't the output become $222 \times 222 \times 30$, or something similar?**

That's a great question!

Remember, our input image has three channels: red, green, and blue. Each convolutional filter processes all three channels together.

In other words, a filter with a spatial size of $3 \times 3$ actually has a complete shape of:

$$3 \times 3 \times 3$$

The filter calculates a weighted sum across the red, green, and blue channels to produce **one single value at each spatial position**.

So, although the input has three channels, each filter produces only one two-dimensional feature map.

Now, how many filters do we have?

```
layers.Conv2D(10, (3, 3), activation="relu")
```

We have 10 filters. Each filter produces one feature map, so we end up with 10 feature maps stacked together.

That's why the final output shape is:

$$\boxed{222 \times 222 \times 10}$$

The first two dimensions come from the spatial size of the feature maps, while the last dimension comes from the number of filters.

**Remember:** The number of input channels determines the depth of each filter, while the number of filters determines the depth of the output.

#### 3. Second Convolutional Block

```
(layers.Conv2D(10, (3, 3), activation="relu"),)
(layers.Conv2D(10, (3, 3), activation="relu"),)
(layers.AveragePooling2D((4, 4)),)
```

The second block continues extracting features from the representations produced by the first block.

- Third convolution: Learns additional patterns from the previously extracted features.
- Fourth convolution: Combines existing features into potentially more informative visual patterns.
- Average pooling: Reduces the spatial dimensions again, making the representation smaller and more manageable for the subsequent dense layers.

The output dimensions are:

| Stage | Output shape |
|---|---|
| Input to block | $(110 \times 110 \times 10)$ |
| Conv2D (10 filters) | $(108 \times 108 \times 10)$ |
| Conv2D (10 filters) | $(106 \times 106 \times 10)$ |
| AveragePooling2D | $(26 \times 26 \times 10)$ |

At this stage, the network has transformed the original image into a collection of 10 feature maps, each measuring $(26 \times 26)$.

#### 4. Flatten Layer

```
layers.Flatten(),
```

The convolutional blocks produce a three-dimensional feature representation. However, a standard `Dense` layer operates on a vector of features.

The `Flatten` layer converts the feature maps into a one-dimensional vector without learning any parameters.

The input to this layer has shape:

$26 \times 26 \times 10$

Therefore, the flattened vector contains:

$26 \times 26 \times 10 = 6,760$

features.

The resulting shape is `(6760,)` for each individual image.

## 5. Hidden Dense Layer

```
layers.Dense(5, activation='relu'),
```

This fully connected layer receives the 6760 features and transforms them into five learned feature combinations.

Every neuron is connected to every input feature. The layer learns weights and biases that help determine which combinations of extracted visual features are useful for classification.

ReLU is applied again to introduce non-linearity.

The layer contains:

$(6760 \times 5) + 5 = 33,805$

trainable parameters, including its weights and biases.

## 6. Output Layer

```
layers.Dense(1, activation='sigmoid')
```

The final layer contains a single neuron because we only have two classes in the dataset.

The sigmoid activation function converts the neuron's output into a value between 0 and 1:

$\sigma(x) = \frac{1}{1+e^{-x}}$

This value can be interpreted as the predicted probability of the positive class, assuming the model is trained with an appropriate binary classification objective.

For example, if the model produces an output of `0.85`, it indicates an estimated probability of 85% for the positive class. A threshold such as `0.5` can be used to assign a class label, although the appropriate threshold may depend on the application.

# Complete Architecture at a Glance

| Layer | Output shape | Purpose |
|---|---|---|
| Input | `224 × 224 × 3` | Accept RGB images |
| Rescaling | `224 × 224 × 3` | Normalize pixel values |
| Conv2D | `222 × 222 × 10` | Extract visual features |
| Conv2D | `220 × 220 × 10` | Learn richer features |
| AveragePooling2D | `110 × 110 × 10` | Reduce spatial dimensions |
| Conv2D | `108 × 108 × 10` | Extract additional features |
| Conv2D | `106 × 106 × 10` | Learn more complex patterns |
| AveragePooling2D | `26 × 26 × 10` | Further reduce dimensions |
| Flatten | `6760` | Convert feature maps to a vector |
| Dense | `5` | Combine learned features |
| Dense | `1` | Predict binary class probability |

AND NOW WE CAN TRAIN THE MODEL.

```
from keras import optimizers, losses, metrics

model.compile(
    optimizer=optimizers.Adam(),
    loss=losses.BinaryCrossentropy(),
    metrics=[metrics.BinaryAccuracy()],
)
```

```
model.fit(
    training_dataset,
    validation_data=validation_dataset,
    epochs=5,
)
```

```
Epoch 1/5

50/50 ━━━━━━━━━━━━━━━━━━━━ 9s 59ms/step - binary_accuracy: 0.6075 - loss: 0.6389 - v
Epoch 2/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - binary_accuracy: 0.7631 - loss: 0.5034 - v
Epoch 3/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - binary_accuracy: 0.8225 - loss: 0.4139 - v
Epoch 4/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - binary_accuracy: 0.8381 - loss: 0.3921 - v
Epoch 5/5
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - binary_accuracy: 0.8637 - loss: 0.3420 - v


<keras.src.callbacks.history.History at 0x72f9859f60f0>
```

Noice!

In just 5 epochs the model is performing well on the training dataset.

And if you've noticed on the code I've passed an extra argument `validation_data` to the `fit` method.

Up until now you guys know that a dataset is split into two parts, the training dataset and the testing dataset.

There is a third part, the validation dataset.

The validation dataset is just like the testing dataset, in fact sometimes the validation dataset is the same as the testing dataset.

This dataset is used to monitor the model's performance on unseen data. And as the model is learning, we can see if the model is overfitting or underfitting.

Now take look at the `val_binary_accuracy` metric and tell me how is the model performing on unseen data.

For me, I think it's perfect. The accuracy is in both the training and validation dataset is close and the loss is also close.

Now, here is a nice resource you visualize how a convolutional network learns: Visualizing the filters of a convolutional neural network

Go to the website and play around a little and also read the whole thing. You'll get a lot of insights that I cannot explain without properly giving you a visual understanding.

And also here's a small task for you take the above model and instead of `average pooling` you can use `max pooling` by replacing `layers.AveragePooling2D((2, 2))` with `layers.MaxPooling2D((2, 2))` and see the difference in the model's performance.

Now our model evaluation is not completed yet. We should properly evaluate the model.

```
1 | model.evaluate(testing_dataset)
```

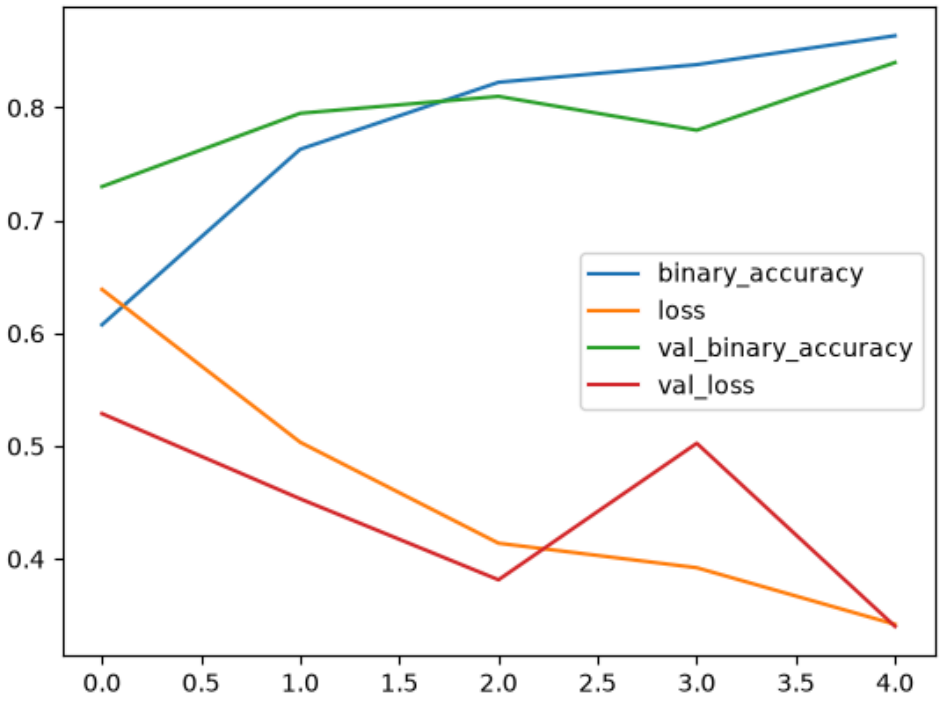
```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - binary_accuracy: 0.8350 - loss: 0.3854


[0.3854452967643738, 0.8349999785423279]
```

In the testing set the model is performing pretty too. Let's see the loss curve.

```
1  import pandas as pd
2
3  pd.DataFrame(model.history.history).plot()
```

```
<Axes: >
```




Awh! would you look at that! accuracy for both the training and the validation dataset is going up and the loss is going down. That is a very good signal that the model is learning and is not overfitting. Also with more epochs the model should learn more.

Let's do a classification report now, because we should see if the model is predicting each class correctly.

```
1  predictions = model.predict(testing_dataset)
2  predictions
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 63ms/step


array([[0.10191924],
       [0.07007429],
       [0.809599  ],
       [0.59127   ],
       [0.94595516],
       [0.48744705],
    ...
       [0.9790647 ],
       [0.9787854 ],
       [0.86135936],
       [0.9590761 ]], dtype=float32)
```

```
predictions = (predictions > 0.5) * 1
predictions
```

```
array([[0],
       [0],
       [1],
       [1],
        ....
       [1],
       [1],
       [1],
       [1]])
```

We can use the numpy broadcasting to just check if a prediction is greater than 0.5 or not. If it is, than it is 1 else it is 0.

Now we have to extract the actual labels from the testing dataset.

```
y_actual = testing_dataset.map(lambda x, y: y)

list(y_actual.as_numpy_iterator())
```

```
2026-10-21 15:53:55.234511: I tensorflow/core/framework/local_rendezvous.cc:407] Loc


[array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32),
 array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32),
 array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32),
 array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int32),
 array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int32),
 array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int32),
 array([1, 1, 1, 1, 1, 1, 1, 1], dtype=int32)]
```

The `map` method can be used to apply a function to all the elements in the dataset.

You have to pass a `map_func` argument which is a function that will be applied to all the elements in the dataset. In our datasets case to all the rows one by one, we will get a new tf.data.Dataset object with the values that we will get after applying the function.

So, that why we can use `as_numpy_iterator` to get the values as a numpy array. And if we convert all the values to a python list we get a list of numpy arrays.

Because of the image dataset being batched we get multiple numpy arrays in the list containing all the labels.

So, we have merge all of them togather to get the full list of labels.

```
1 | y_actual = np.concatenate(list(y_actual.as_numpy_iterator()))
2 | y_actual
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1], dtype=int32)
```

And we have all the

And now comes the part where I anwer why I kept the `validation_data` and `test_data` unshuffled.

So, the issue is not the model or the predictions. It is related to how the data is loaded using `tf.data.Dataset` object.

As I said earlier the `tf.data.Dataset` object is a lazy object. It will only load the data when it is needed.

So, everytime I use `testing_dataset` the data will load and be converted to a numpy array. But if that process the data will be shuffled everytime.

So, first I get the predictions, that's when the data is loaded and shuffled. Then when I get the `actual labels` that's when the data is loaded and shuffled again.

So, the prediction and the actual labels will get `out of sync`. So, there will be no way compare the prediction with the actual labels.

I hope you know for scikitleanrs classification report is just comparing the predicted labels with the actual labels by index.

So, that's why I kept the `validation_data` and `test_data` unshuffled.

> Rule of Thumb: When lazy loading a dataset, shuffle the training set but never shuffle the validation set or the test set because the data will go out of sync while evaluating the model. That'll be bad!!

So, the model is not overfitting and performing well after being trained for only 5 epochs.

Let's set a base line and see if we can improve the performce of the model even more in just 5 epochs.

# Improving The models performance

There are many ways we can try to improve a models performance. Some of them are:

1. Standardization/Normalization.
2. Train for more epochs.
3. More complex models. DUH?!!
4. Different activation functions.
5. Different optimizers.

And many more. But when you are trying to improve the performance of a convolutional neural network you can do another very special thing.

## Data Augmentation

`Data augmentation` is a technique used artificially explan a training set, increaing both the size and the diversity while preserving the original data distribution.

Data augmentation can be used to improve the `generalization` power of a model.

It would be best if I show you what it is.

```
1 | rotation = layers.RandomRotation(0.2)
```

Data augmentation classes are found in the `layers` module. And in the above cell I made a `RandomRotation` object using the `layers.RandomRotation` class. Inside that class we can pass the angle of rotation. I passed .2 which means 20 degrees.

So, each image passed to the model will be rotated by a random angle between -20 and 20 degrees.

Let's see if it works!

> You can use the instance of the class like a function.

```
images, labels = training_dataset.as_numpy_iterator().next()

ind = random.randint(0, len(images) - 1)

plt.subplot(1, 2, 1)
plt.imshow(images[ind] / 255)
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(rotation(images[ind]) / 255)
plt.axis("off")
plt.show()
```



As you can see the images are the same. But the second image is rotated by a random angle between -20 and 20 degrees.

There are other data augmentation classes in the `layers` module.

- `RandomFlip` : Flips the image horizontally or vertically.
- `RandomTranslation` : Translates the image.
- `RandomZoom` : Zooms the image.

You can explore all of this layers here

One cool thing I want to show you is, we can make a pipeline of data augmentation useing the `sequential` class.

And then see the final result.

```
augmentation_pipeline = tf.keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(factor=0.15, fill_mode="reflect"),
        layers.RandomZoom(height_factor=0.1, width_factor=0.1),
        layers.RandomBrightness(factor=0.2),
    ]
)
```

```
images, labels = training_dataset.as_numpy_iterator().next()

ind = random.randint(0, len(images) - 1)

plt.subplot(1, 2, 1)
plt.imshow(images[ind] / 255)
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(augmentation_pipeline(images[ind]) / 255)
plt.axis("off")
plt.show()
```





Because the augmentation pipeline applies random transformations on-the-fly during each epoch, the probability of the model encountering the exact same image configuration twice is extremely low. Consequently, during every training iteration, the network is continuously exposed to unique variations and unique perspectives of the training data.

So, even though the model is trained on the same data, it learns to recognize the same things but in different ways.

But there is a problem with this approach.

The model will generalize well but the learning process will be very slow.

So, it's a `necessary-evil` trade-off.

So, let's make a model with data augmentation and see if we can improve the model's performance.

```python
model2 = models.Sequential(
    [
        layers.Input((224, 224, 3)),
        # augmentation layers
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(factor=0.15, fill_mode="reflect"),
        layers.RandomZoom(height_factor=0.1, width_factor=0.1),
        layers.RandomBrightness(factor=0.2),
        # convolutional layers
        layers.Rescaling(1.0 / 255),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((4, 4)),
        layers.Flatten(),
        layers.Dense(5, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ]
)

model.summary()
```

**Model: "sequential_1"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| rescaling (Rescaling) | (None, 224, 224, 3) | 0 |
| conv2d_1 (Conv2D) | (None, 222, 222, 10) | 280 |
| conv2d_2 (Conv2D) | (None, 220, 220, 10) | 910 |
| average_pooling2d_1 (AveragePooling2D) | (None, 110, 110, 10) | 0 |
| conv2d_3 (Conv2D) | (None, 108, 108, 10) | 910 |
| conv2d_4 (Conv2D) | (None, 106, 106, 10) | 910 |
| average_pooling2d_2 (AveragePooling2D) | (None, 26, 26, 10) | 0 |
| flatten_1 (Flatten) | (None, 6760) | 0 |
| dense_3 (Dense) | (None, 5) | 33,805 |
| dense_4 (Dense) | (None, 1) | 6 |

```
Total params: 110,465 (431.51 KB)
```

```
Trainable params: 36,821 (143.83 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

```
Optimizer params: 73,644 (287.68 KB)
```

The above model is the same as the previous one but with data augmentation.

Another nice thing I want to show you before training the model again is. We can combine multiple `Sequential` models inside another `Sequential` model. Best practice is to separate the data augmentation and the convolutional layers.

We can do that like this:

```
data_augmentation = tf.keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(factor=0.15, fill_mode="reflect"),
        layers.RandomZoom(height_factor=0.1, width_factor=0.1),
        layers.RandomBrightness(factor=0.2),
    ]
)

model3 = models.Sequential(
    [
        layers.Input((224, 224, 3)),
        data_augmentation,
        layers.Rescaling(1.0 / 255),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((4, 4)),
        layers.Flatten(),
        layers.Dense(5, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ]
)

model3.summary()
```

```
Model: "sequential_5"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| sequential_4 (Sequential) | (None, 224, 224, 3) | 0 |
| rescaling_2 (Rescaling) | (None, 224, 224, 3) | 0 |
| conv2d_9 (Conv2D) | (None, 222, 222, 10) | 280 |
| conv2d_10 (Conv2D) | (None, 220, 220, 10) | 910 |
| average_pooling2d_5 (AveragePooling2D) | (None, 110, 110, 10) | 0 |
| conv2d_11 (Conv2D) | (None, 108, 108, 10) | 910 |
| conv2d_12 (Conv2D) | (None, 106, 106, 10) | 910 |
| average_pooling2d_6 (AveragePooling2D) | (None, 26, 26, 10) | 0 |
| flatten_3 (Flatten) | (None, 6760) | 0 |
| dense_7 (Dense) | (None, 5) | 33,805 |
| dense_8 (Dense) | (None, 1) | 6 |

**Total params:** 36,821 (143.83 KB)

**Trainable params:** 36,821 (143.83 KB)

**Non-trainable params:** 0 (0.00 B)

This way the model architecture stays clean and we can focus on the data augmentation and testing it as well.

So, let's train.

```
model3.compile(
    optimizer=optimizers.Adam(),
    loss=losses.BinaryCrossentropy(),
    metrics=[metrics.BinaryAccuracy()],
)

model3.fit(
    training_dataset,
    validation_data=validation_dataset,
    epochs=10,
)
```

```
Epoch 1/10


/home/itvaya/miniforge3/envs/ml_env/lib/python3.12/site-packages/keras/src/trainers/
  self.data_adapter = data_adapters.get_data_adapter(


50/50 ━━━━━━━━━━━━━━━━━━━━ 4s 26ms/step - binary_accuracy: 0.6750 - loss: 0.6153 - v
Epoch 2/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.7013 - loss: 0.5777 - v
Epoch 3/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.7319 - loss: 0.5429 - v
Epoch 4/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - binary_accuracy: 0.7531 - loss: 0.5109 - v
Epoch 5/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.7962 - loss: 0.4682 - v
Epoch 6/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.8219 - loss: 0.4055 - v
Epoch 7/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.8350 - loss: 0.3808 - v
Epoch 8/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.8375 - loss: 0.3730 - v
Epoch 9/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.8525 - loss: 0.3468 - v
Epoch 10/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - binary_accuracy: 0.8556 - loss: 0.3313 - v


<keras.src.callbacks.history.History at 0x72f9855bb140>
```

Nice! The model is learning pretty well and in the validation set the accuracy is increasing as well.

And I ran the model for 10 epochs because as I said earlier data augmentation is a `necessary-evil` trade-off. So, if I had ran it for 5 epochs the performace would not have been good at all. You can clearly see that from the training epochs.

OKHAY!! THE MODEL IS TRAINED MAYBE NOT THAT GOOD BUT IT IS LEARNING.

So, let's do something different.

Let's try to predict an image that is not in any dataset.



I downloaded the above image. I love cheese-cakes so let's see if the model can predict it correct.

But first we have to load the image.

We can do it in many ways.

```
cheesecake = plt.imread("cheesecake.jpeg")
cheesecake
```

```
array([[[166, 190, 218],
        [166, 190, 218],
        [166, 190, 218],
        ...,
        ...,
        ...,
        [225, 225, 233],
        [209, 205, 219],
        [186, 182, 196]]], dtype=uint8)
```

`plt.imread` is a function that reads an image from a file and returns it as a numpy array.

We should see the shape of image.

```
cheesecake.shape
```

```
(597, 514, 3)
```

OWH? that's a big image. But the model only accepts images with a shape of `(224, 224, 3)`.

So, have to resize the image and that will be a hassle to do with numpy. But lucky for us tensorflow has a built-in module for image processing called `tf.image`.

It can do a lot of things. So, instead of loading the image as a numpy array we can load it and resize it as a tensor.

```
img_bytes = tf.io.read_file("cheesecake.jpeg")
img_bytes
```

```
<tf.Tensor: shape=(), dtype=string, numpy=b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01
```

We first have to read the image from a file.

Now we load the image as a tensor.

```
cheesecake = tf.image.decode_jpeg(img_bytes, channels=3)
cheesecake
```

```
<tf.Tensor: shape=(597, 514, 3), dtype=uint8, numpy=
array([[[166, 190, 216],
        [166, 190, 216],
        [165, 189, 215],
        ...,
        ...,
        ...,
        [222, 225, 230],
        [209, 205, 219],
        [186, 182, 196]]], dtype=uint8)>
```

There! successfully loaded. And now we can easily resize it.

```
cheesecake = tf.image.resize(cheesecake, (224, 224))
cheesecake
```

```
<tf.Tensor: shape=(224, 224, 3), dtype=float32, numpy=
array([[[166.     , 190.     , 216.     ],
        [165.     , 189.     , 215.     ],
        [164.     , 188.     , 214.     ],
        ...,
        [ 69.2366 ,  99.2366 , 123.2366 ],
        [ 68.     ,  98.     , 122.     ],
        [ 66.     ,  96.     , 120.     ]],

         ....

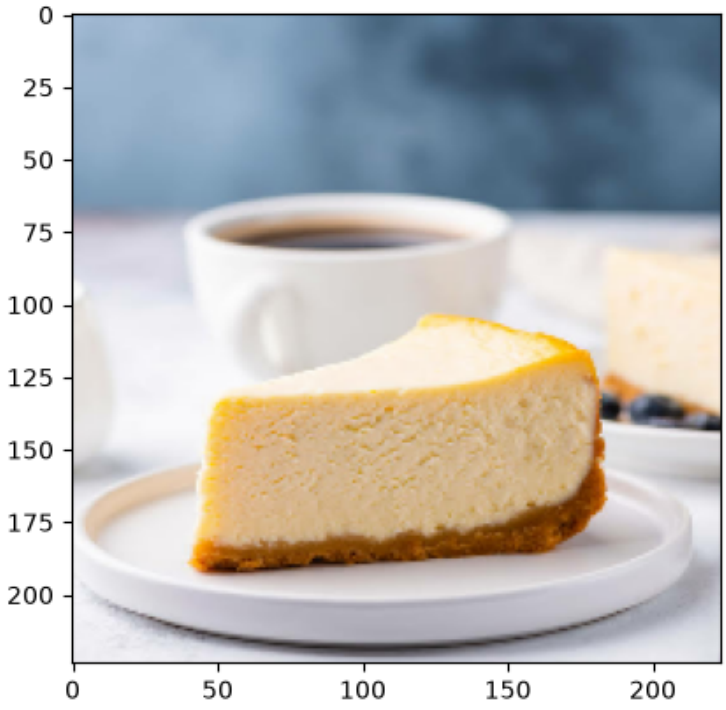
       [[239.     , 238.     , 243.     ],
        [239.     , 238.     , 243.     ],
        [239.     , 238.     , 243.     ],
        ...,
        [222.     , 225.     , 230.     ],
        [222.     , 225.     , 230.     ],
        [203.09146, 199.09146, 213.09146]]], dtype=float32)>
```

Yeah. It's that easy.

Let's see how our image looks like after resizing.

```
1 | plt.imshow(cheesecake / 255)
```

```
<matplotlib.image.AxesImage at 0x72f9791d3620>
```




Exactly as before but now with a shape of `(224, 224, 3)`. Cool!

I'm going to make the above resizing step a function because we might just need it later.

```
def load_and_resize(filepath, image_shape=(224, 224)):
    image = tf.io.read_file(filepath)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, image_shape)
    return image
```

So, let's predict the image. We know this is a cheesecake but let's see if our model learned it.

```
model_input = tf.expand_dims(cheesecake, axis=0)
model_input.shape
```

```
TensorShape([1, 224, 224, 3])
```

We have to do this because the model expects a batch of images. So, have to put the image inside an array.

now let's predict.

```
predictions = model3.predict(model_input)
predictions
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step


array([[0.0907029]], dtype=float32)
```

Now we get the probability distribution of the image being `1`. I told you earlier this value reflects the index of the class. If the value is less than `0.5` the image class is `index 0`. If the value is greater than `0.5` the image class is `index 1`.

Let's see what was our classes.

```
training_dataset.class_names
```

```
['cheesecake', 'pizza']
```

So, Now, we just have to do a comparison and that will be the predicted class index.

```
training_dataset.class_names[int(tf.round(predictions[0][0]))]
```

```
'cheesecake'
```

OOOOOOWWWWWWW!!!!!!

That is NOICEEEE!! perfect prediction from the model.

Let's break down the code.

`training_dataset.class_names` is a list of class names. And `int(tf.round(predictions[0][0]))` is the predicted class index.

`tf.round` will automatically round the value to the nearest integer, which is the exact comparison we need. If the value is less than `0.5` it'll be `0`. If the value is greater than `0.5` it'll be `1`.

But this will return a tensor of `float32` so after we convert it we can index from the `class_names` list and what do we get?? You guessed it! The predicted class name.

And the model is predicting the class very nicely.

That good.

Now how about we try the same thing in multi-class classification?

# CNN Multi-class Classification

We can follow the same steps we followed in the previous sections:

- Load the Data.
- Preprocess the data.
- Create a model.
- Fit the model.
- Evaluate.
- Try to improve the model by fine tuning different parts of it.

So, first loading the data.

Just like before I've already made a split of the `food 101` dataset and randomly selected 10 classes. You can run the following code to download the whole zip and decompress it.

```
# !wget "https://github.com/RishatTalukder/Deep_Learning_With_Tensorflow/raw/refs/heads/main/

# import zipfile

# zip_ref = zipfile.ZipFile('10FoodClasses.zip')
# zip_ref.extractall(path='resource')
# zip_ref.close()
```

Now Let's take a look at the `10FoodClasses` folder like before

```
import os

for dirpath, dirnames, filenames in os.walk("resource/10FoodClasses"):
    print(
        f"There are {len(dirnames)} directories and {len(filenames)} images in '{dirpath}'."
    )
```

```
There are 2 directories and 0 images in 'resource/10FoodClasses'.
There are 10 directories and 0 images in 'resource/10FoodClasses/train'.
There are 0 directories and 250 images in 'resource/10FoodClasses/train/chicken_wing
There are 0 directories and 250 images in 'resource/10FoodClasses/train/tiramisu'.
There are 0 directories and 250 images in 'resource/10FoodClasses/train/hamburger'.
There are 0 directories and 250 images in 'resource/10FoodClasses/train/mussels'.
There are 0 directories and 250 images in 'resource/10FoodClasses/train/cannoli'.
There are 0 directories and 250 images in 'resource/10FoodClasses/train/chocolate_ca
There are 0 directories and 250 images in 'resource/10FoodClasses/train/bread_puddir
There are 0 directories and 250 images in 'resource/10FoodClasses/train/ramen'.
There are 0 directories and 250 images in 'resource/10FoodClasses/train/miso_soup'.
There are 0 directories and 250 images in 'resource/10FoodClasses/train/creme_brulee
There are 10 directories and 0 images in 'resource/10FoodClasses/val'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/chicken_wings'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/tiramisu'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/hamburger'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/mussels'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/cannoli'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/chocolate_cake'
There are 0 directories and 50 images in 'resource/10FoodClasses/val/bread_pudding'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/ramen'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/miso_soup'.
There are 0 directories and 50 images in 'resource/10FoodClasses/val/creme_brulee'.
```

Alright! 800 images in each food class in the training set and 200 images in each food class in the validation set.

You might ask where is the testing set.

It's the same thing, to be honest. Validation set is used to see performance on the unseen data while training the model and testing set is used to see performance on the unseen data after the model is trained.

Both are unseen datasets. So, We can use it interchangeably.

I'll be using `val` set for both validation and testing.

So, let's load the data.

```python
from keras.utils import image_dataset_from_directory

training_set = image_dataset_from_directory(
    directory="resource/10FoodClasses/train",
    batch_size=24,
    image_size=(224, 224),
    shuffle=True,
    seed=42,
)

validation_set = image_dataset_from_directory(
    directory="resource/10FoodClasses/val",
    batch_size=24,
    image_size=(224, 224),
    shuffle=False,
)
```

```
2026-10-25 15:59:50.773290: I tensorflow/core/platform/cpu_feature_guard.cc:210] Thi
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorF


Found 2500 files belonging to 10 classes.


WARNING: All log messages before absl::InitializeLog() is called are written to STDE
I0000 00:00:1792922392.422969  214433 gpu_device.cc:2020] Created device /job:localh


Found 500 files belonging to 10 classes.
```

Wow, now we have 10000 images,(I just hope my pc stays alive).

So, let's do this.

What classes do we have?

```python
training_set.class_names
```

```
['bread_pudding',
 'cannoli',
 'chicken_wings',
 'chocolate_cake',
 'creme_brulee',
 'hamburger',
 'miso_soup',
 'mussels',
 'ramen',
 'tiramisu']
```

So, nice diverse classes.

So, let's make a model. This time we have to keep in my our model have to predict `10 classes` instead of just 2. So, the model outpput layer will be a little different.

If you don't know about multi-class classification, you can read one of the previous article on multi-class classification

Mainly there onely one thing we have to do.

- The number of neurons in the output layer must be equal to the number of classes.

and the activation function for the output layer will be `softmax` instead of `sigmoid`.

So, let's make a model.

```
from keras import layers, activations, models


model4 = models.Sequential(
    [
        layers.Input((224, 224, 3)),
        layers.Rescaling(1.0 / 255),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((4, 4)),
        layers.Flatten(),
        layers.Dense(5, activation="relu"),
        layers.Dense(10, activation=activations.softmax),
    ]
)

model4.summary()
```

```
Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| rescaling (Rescaling) | (None, 224, 224, 3) | 0 |
| conv2d (Conv2D) | (None, 222, 222, 10) | 280 |
| conv2d_1 (Conv2D) | (None, 220, 220, 10) | 910 |
| average_pooling2d (AveragePooling2D) | (None, 110, 110, 10) | 0 |
| conv2d_2 (Conv2D) | (None, 108, 108, 10) | 910 |
| conv2d_3 (Conv2D) | (None, 106, 106, 10) | 910 |
| average_pooling2d_1 (AveragePooling2D) | (None, 26, 26, 10) | 0 |
| flatten (Flatten) | (None, 6760) | 0 |
| dense (Dense) | (None, 5) | 33,805 |
| dense_1 (Dense) | (None, 10) | 60 |

**Total params:** 36,875 (144.04 KB)

**Trainable params:** 36,875 (144.04 KB)

**Non-trainable params:** 0 (0.00 B)

Model 4 is basically `model 3` with the output layer changed to fit our requirement.

As this model was performing pretty good on cheesecake and pizza dataset. Let's how good it handles 10 food classes.

So, let's compile and fit the model.

```
from keras import optimizers, losses, metrics

model4.compile(
    optimizer=optimizers.Adam(),
    loss=losses.SparseCategoricalCrossentropy(),
    metrics=[metrics.SparseCategoricalAccuracy()],
)
```

Now, I'll use 50% the validation set as validation data whole the model is training we can do that by setting `validation_split` to 0.5.

```
model4.fit(
    training_set,
    epochs=10,
    validation_data=validation_set,
)
```

```
Epoch 1/10


/home/itvaya/miniforge3/envs/ml_env/lib/python3.12/site-packages/keras/src/trainers/
  self.data_adapter = data_adapters.get_data_adapter(
2026-10-25 15:59:54.461992: I external/local_xla/xla/service/service.cc:163] XLA ser
2026-10-25 15:59:54.462010: I external/local_xla/xla/service/service.cc:171]   Strea
2026-10-25 15:59:54.700813: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_ut
2026-10-25 15:59:55.119674: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.c
2026-10-25 15:59:56.062400: I external/local_xla/xla/stream_executor/cuda/subprocess

2026-10-25 15:59:56.083284: I external/local_xla/xla/stream_executor/cuda/subprocess



 12/105 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - loss: 2.3003 - sparse_categorical_accura

I0000 00:00:1792922400.093280  214506 device_compiler.h:196] Compiled cluster using


105/105 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 2.2715 - sparse_categorical_accura

2026-10-25 16:00:05.630571: I external/local_xla/xla/stream_executor/cuda/subprocess

2026-10-25 16:00:05.683334: I external/local_xla/xla/stream_executor/cuda/subprocess

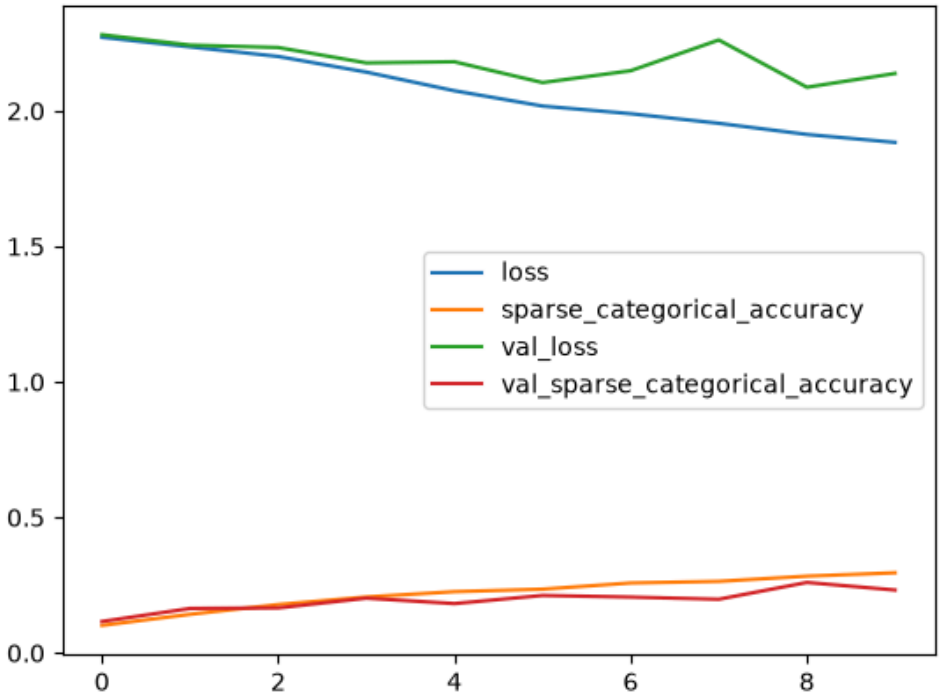


105/105 ━━━━━━━━━━━━━━━━━━━━ 13s 62ms/step - loss: 2.2715 - sparse_categorical_accur
Epoch 2/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 2.2360 - sparse_categorical_accura
Epoch 3/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 2.2007 - sparse_categorical_accura
Epoch 4/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 2.1430 - sparse_categorical_accura
Epoch 5/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 2.0741 - sparse_categorical_accura
Epoch 6/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 2.0178 - sparse_categorical_accura
Epoch 7/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 1.9896 - sparse_categorical_accura
Epoch 8/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 1.9541 - sparse_categorical_accura
Epoch 9/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 1.9130 - sparse_categorical_accura
Epoch 10/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 1.8837 - sparse_categorical_accura
```

```
<keras.src.callbacks.history.History at 0x794751362ff0>
```

```
1 import pandas as pd
2
3 pd.DataFrame(model4.history.history).plot()
```

```
<Axes: >
```




Umph! That is not good. The model is learning because the loss is decreasing in the training set but in the validation set it is increasing and decreasing at the same time.

This is one of the best examples of `overfitting`. The model is learning the patterns of the training set but on unseen data it is performing really really bad.

So, we must fix this as fast as can. So, what can we do to reduce overfitting?

- **Increase the size of the training set**: More data more patterns.
- **Make the model a little less complex**: Yes this is a cause of overfitting. Right now the model has 9 layers and 6 of those a convolution layers. Maybe the model is learning some patterns from the training set that are not there in the validation set.
- **Data Augmentation**: We can change the models input by changing the angle, brightness, contrast, etc. This way the model will not learn repeated patterns and generalize even better like I've shown in the previous section of this article.

Now, I dont have another dataset lying around with more data, so the other two options are my only way.

# Training a Simpler Model

I remove some layers from the model and let's see if it performs better.

```
model5 = models.Sequential(
    [
        layers.Input((224, 224, 3)),
        layers.Rescaling(1.0 / 255),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((4, 4)),
        layers.Flatten(),
        # layers.Dense(5, activation='relu'),
        layers.Dense(10, activation=activations.softmax),
    ]
)

model5.summary()
```

**Model: "sequential_9"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| rescaling_9 (Rescaling) | (None, 224, 224, 3) | 0 |
| conv2d_20 (Conv2D) | (None, 222, 222, 10) | 280 |
| average_pooling2d_6 (AveragePooling2D) | (None, 111, 111, 10) | 0 |
| conv2d_21 (Conv2D) | (None, 109, 109, 10) | 910 |
| average_pooling2d_7 (AveragePooling2D) | (None, 27, 27, 10) | 0 |
| flatten_9 (Flatten) | (None, 7290) | 0 |
| dense_16 (Dense) | (None, 10) | 72,910 |

**Total params:** 74,100 (289.45 KB)

**Trainable params:** 74,100 (289.45 KB)

**Non-trainable params:** 0 (0.00 B)

Now, I made a much simpler model with less layers and I even removed the `Dense` before the output layer.

Let's see if it performs better.

```
model5.compile(
    optimizer=optimizers.Adam(),
    loss=losses.SparseCategoricalCrossentropy(),
    metrics=[metrics.SparseCategoricalAccuracy()],
)

model5.fit(
    training_set,
    epochs=10,
    validation_data=validation_set,
)
```

```
Epoch 1/10


/home/itvaya/miniforge3/envs/ml_env/lib/python3.12/site-packages/keras/src/trainers/
  self.data_adapter = data_adapters.get_data_adapter(


105/105 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - loss: 2.1806 - sparse_categorical_accura
Epoch 2/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 1.8814 - sparse_categorical_accurac
Epoch 3/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 1.6850 - sparse_categorical_accurac
Epoch 4/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 1.5103 - sparse_categorical_accurac
Epoch 5/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 1.3389 - sparse_categorical_accurac
Epoch 6/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 1.1488 - sparse_categorical_accurac
Epoch 7/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9306 - sparse_categorical_accurac
Epoch 8/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.7892 - sparse_categorical_accurac
Epoch 9/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.5964 - sparse_categorical_accurac
Epoch 10/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.4704 - sparse_categorical_accurac

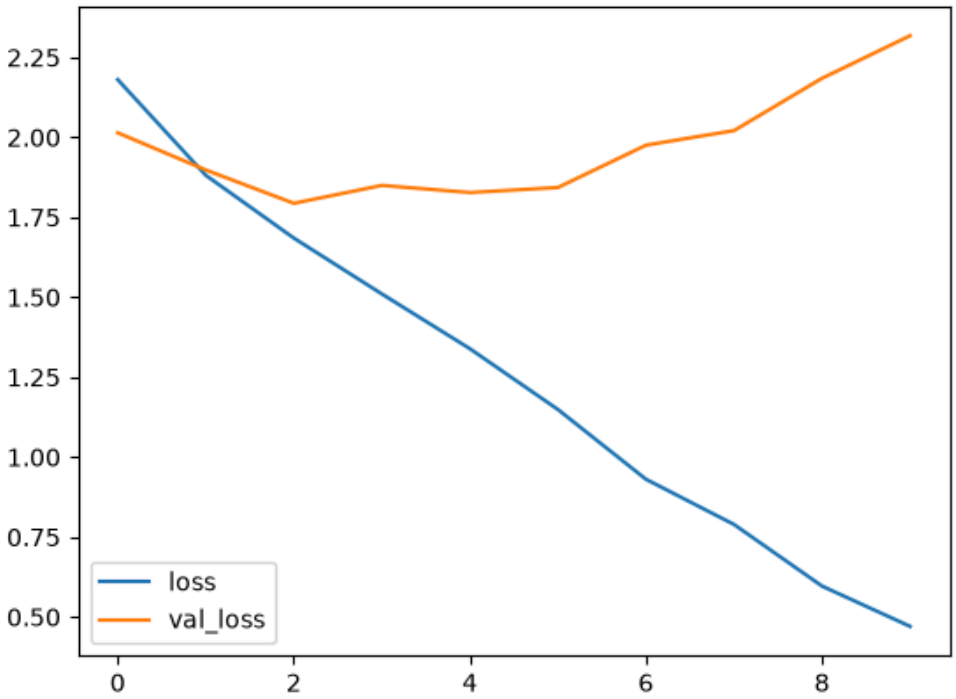

<keras.src.callbacks.history.History at 0x7943f584db50>
```

Ow wow! the model is learning very fast. That was supposed to happen i guess. The model is smaller now so it should be learning faster.

Let's see if the loss on validation set is decreasing or increasing.

```
pd.DataFrame(
    data={
        "loss": model5.history.history["loss"],
        "val_loss": model5.history.history["val_loss"],
    }
).plot()
```

```
<Axes: >
```




Nope, still heavily overfitting. The loss on training set is decreasing very fast but the loss on validation set is increasing, not good at all...

So, a less complex model is not helping us here. So, let's try with data augmentation.

```
augmentation = models.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(factor=0.15, fill_mode="reflect"),
        layers.RandomZoom(height_factor=0.1, width_factor=0.1),
        layers.RandomBrightness(factor=0.2),
    ]
)
```

This is the exact same augmentation I used in the previous section of this article.

So, in the simpler model. I'll just add this augmentation so, when the model is training it can learn more patterns from the data.

```
model6 = models.Sequential(
    [
        layers.Input((224, 224, 3)),
        augmentation,
        layers.Rescaling(1.0 / 255),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((2, 2)),
        layers.Conv2D(10, (3, 3), activation="relu"),
        layers.AveragePooling2D((4, 4)),
        layers.Flatten(),
        # layers.Dense(5, activation='relu'),
        layers.Dense(10, activation=activations.softmax),
    ]
)

model6.summary()
```

**Model: "sequential_12"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| sequential_10 (Sequential) | (None, 224, 224, 3) | 0 |
| rescaling_11 (Rescaling) | (None, 224, 224, 3) | 0 |
| conv2d_24 (Conv2D) | (None, 222, 222, 10) | 280 |
| average_pooling2d_10 (AveragePooling2D) | (None, 111, 111, 10) | 0 |
| conv2d_25 (Conv2D) | (None, 109, 109, 10) | 910 |
| average_pooling2d_11 (AveragePooling2D) | (None, 27, 27, 10) | 0 |
| flatten_11 (Flatten) | (None, 7290) | 0 |
| dense_18 (Dense) | (None, 10) | 72,910 |

**Total params:** 74,100 (289.45 KB)

**Trainable params:** 74,100 (289.45 KB)

**Non-trainable params:** 0 (0.00 B)

Now, time to train.

```
model6.compile(
    optimizer=optimizers.Adam(),
    loss=losses.SparseCategoricalCrossentropy(),
    metrics=[metrics.SparseCategoricalAccuracy()],
)

model6.fit(training_set, epochs=10, validation_data=validation_set)
```

```
Epoch 1/10


/home/itvaya/miniforge3/envs/ml_env/lib/python3.12/site-packages/keras/src/trainers/
  self.data_adapter = data_adapters.get_data_adapter(


105/105 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - loss: 2.1730 - sparse_categorical_accura
Epoch 2/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.9696 - sparse_categorical_accura
Epoch 3/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.8880 - sparse_categorical_accura
Epoch 4/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.8485 - sparse_categorical_accura
Epoch 5/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.8236 - sparse_categorical_accura
Epoch 6/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.7920 - sparse_categorical_accura
Epoch 7/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.7864 - sparse_categorical_accura
Epoch 8/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 1.7853 - sparse_categorical_accura
Epoch 9/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.7463 - sparse_categorical_accura
Epoch 10/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 1.7454 - sparse_categorical_accura

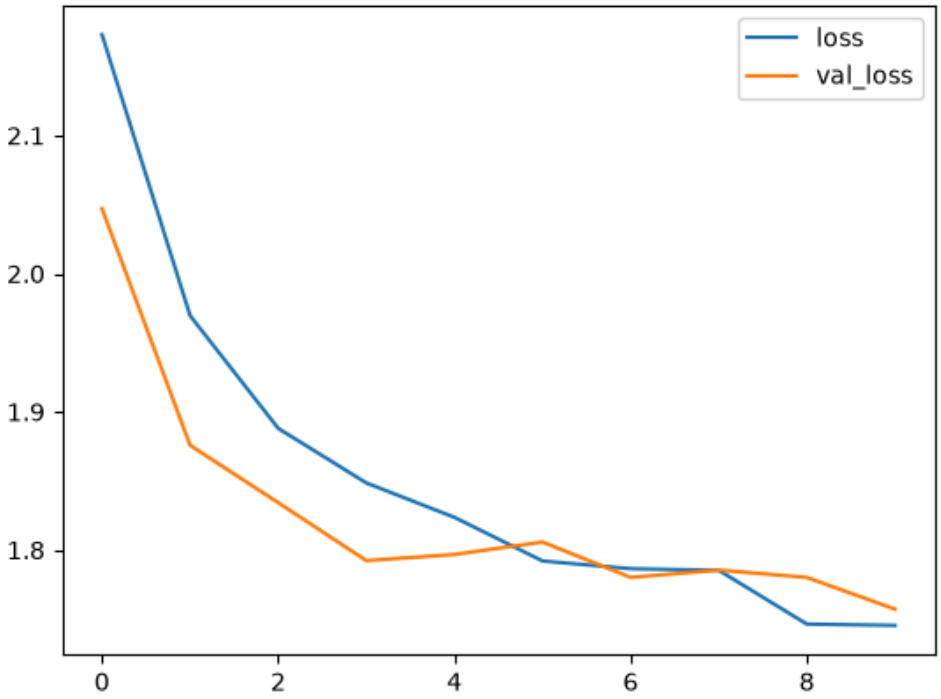

<keras.src.callbacks.history.History at 0x7943f7081be0>
```

Well, now the model is not learning fast like model 5 but if you take a look at the validation loss values those are not increasing either.

Let's see the loss curves.

```
pd.DataFrame(
    data={
        "loss": model6.history.history["loss"],
        "val_loss": model6.history.history["val_loss"],
    }
).plot()
```

```
<Axes: >
```




Would you look at that. The validation loss is decreasing side by side with training loss. So, now we can say our model is actually learning. Hooray! clap clap clap!

The model has learned something from the data and it is also generalizing well. So, no overfitting. That's a win!

And that's why we do data augmentation. It kinda forces a model to learn new patterns and reduce repeated patterns. this is great.

The augmented model behaves differently from the previous model. Its training and validation losses remain relatively close, and the validation loss generally trends downward. This suggests that the model is not showing the same obvious overfitting pattern as before.

However, its accuracy is still only around 39% on both the training and validation sets after 10 epochs. We cannot conclude that the model is performing well simply because the two loss curves are close together.

The model may benefit from further training, a different architecture, or other hyperparameter adjustments. We should evaluate these changes using validation performance and reserve the test set for a final, unbiased evaluation.

There are some other things we could've done to improve the model. I want you guys to try those things out.

- **Change learning rate**: The default learning rate is 0.001. You could try 0.01 or 0.1 or less than 0.001.

- **Change batch size**: while loading I set the batch size to 24. You could try 32 or 64.
- **Bigger or smaller image size**: I set the image size to 224x224. You could try something bigger or smaller and see how good the model performs.

So, one final thing I want to do is test the model on a custom image like I did in the binary classification example.

Let's see what classes we are working with.

```
1 | training_set.class_names
```

```
['bread_pudding',
 'cannoli',
 'chicken_wings',
 'chocolate_cake',
 'creme_brulee',
 'hamburger',
 'miso_soup',
 'mussels',
 'ramen',
 'tiramisu']
```

I choose hamburger as the custom image.




Let's see what our model thinks of this image.

I'll use the `load and resize` function from the previous section.

```
import tensorflow as tf


def load_and_resize(filepath, image_shape=(224, 224)):
    image = tf.io.read_file(filepath)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, image_shape)
    return image
```

```
from matplotlib import pyplot as plt

hamburger = load_and_resize(filepath="hamburger.png")

plt.imshow(hamburger / 255)
plt.axis("off")
```

```
(np.float64(-0.5), np.float64(223.5), np.float64(223.5), np.float64(-0.5))
```



Noice!!

Now, time to predict.

```
prediction = model6.predict(tf.expand_dims(hamburger, axis=0))
prediction
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 148ms/step

array([[0.21988583, 0.01411572, 0.21898842, 0.00447429, 0.12252687,
        0.19721405, 0.05967672, 0.00893284, 0.1215772 , 0.03260802]],
      dtype=float32)
```

We get the probability distribution for each class. let's see which class has the highest probability.

```python
import numpy as np

# loping through the labels and printing the predicted probabilities
for i, label in enumerate(training_set.class_names):
    print(f"{label}: {prediction[0][i] * 100:.2f}%")


print(
    f"The predicted label is: {training_set.class_names[np.argmax(prediction)]}({np.max(predi
)
```

```
bread_pudding: 21.99%
cannoli: 1.41%
chicken_wings: 21.90%
chocolate_cake: 0.45%
creme_brulee: 12.25%
hamburger: 19.72%
miso_soup: 5.97%
mussels: 0.89%
ramen: 12.16%
tiramisu: 3.26%
The predicted label is: bread_pudding(21.99%)
```

Welp! the model predicted wrong. It classified the image of a hamburger as a bread pudding. What does a bread pudding looks like?




Ummm, not much similer to a hamburger. But i think the model is getting influenced by the color of the images. Because if I do some data analytics on the probabilities distribution and sort the classes by the highest probabilities:

```
probabilities = pd.DataFrame(
    data={"label": training_set.class_names, "probability": prediction[0] * 100}
)

probabilities.sort_values(by="probability", ascending=False)
```

| | label | probability |
|---|---|---|
| **0** | bread_pudding | 21.988583 |
| **2** | chicken_wings | 21.898842 |
| **5** | hamburger | 19.721405 |
| **4** | creme_brulee | 12.252687 |
| **8** | ramen | 12.157720 |
| **6** | miso_soup | 5.967672 |
| **9** | tiramisu | 3.260803 |
| **1** | cannoli | 1.411572 |
| **7** | mussels | 0.893284 |

| | label | probability |
|---|---|---|
| **3** | chocolate_cake | 0.447429 |

First three are `bread pudding`, `chicken wings` and then `hamburger`. Hamburger is in the top three and the probabilities are very close. What do these have in common?

The golden brown color.

So, that's it.

I hope this helped you understand the basics of Convolutional Neural Networks and one final thing I'm going to save this model so that maybe if I have time later I can use it again.

```
model6.save("models/model6.keras")
```

# Final Words

This world of Machine learning is very interesting. In my opinion, this is the only field in computer science that mixes two of my favorite things so closely. MATHS and PROGRAMMING. It is a field that is constantly evolving and changing the world unlike anything before. So, I hope my articles can help you learn and understand this ever-changing world of Machine Learning.

If you like my articles, show support by leaving a like and subscribe to my youtube channel

Happy Coding!